EXERCICE ALLOCINE

Application web de gestion de films

Projet individuel

Vue d'ensemble

Vous devez concevoir et développer une application web de gestion de films inspirée d'AlloCiné. Le projet couvre l'ensemble du cycle de développement : mise en place de l'environnement, modélisation de la base de données, développement back-end orienté objet, authentification des utilisateurs et, en option, intégration NoSQL et soins apportés à l'interface.

Composant	Technologie	Statut
Backend	PHP 8.x — Orienté Objet	■ Obligatoire
Base de données	MySQL 8	■ Obligatoire
Conteneurisation	Docker + Docker Compose	■ Obligatoire
Authentification	Sessions PHP	■ Obligatoire
NoSQL	MongoDB	■ Optionnel
Frontend	CSS / JavaScript	■ Optionnel

Contraintes PHP Orienté Objet

L'intégralité de la logique applicative doit être développée en PHP orienté objet. Les éléments suivants sont attendus sans exception :

- **Classes :** User, Film, Genre (et Avis si MongoDB activé)
- **Propriétés :** attributs privés ou protégés
- **Méthodes :** publiques pour les actions, statiques pour les requêtes en base
- **Constructeur :** __construct() pour initialiser les objets
- **PDO :** classe de connexion ou singleton pour centraliser l'accès à la base de données

L'architecture MVC (Model-View-Controller) est optionnelle.

Structure du projet

Organisez votre projet avec une arborescence claire. Le dossier backend/public/ doit contenir le point d'entrée de l'application. Un dossier mysql/init/ doit contenir le script SQL d'initialisation. Si vous adoptez

l'architecture MVC, séparez les dossiers **Controller**, **Model** et **View** dans un dossier src/.

Configuration Docker

Vous devez créer un fichier docker-compose.yml qui orchestre les services suivants :

- **app** — conteneur PHP exposé sur le port **8080**, avec dépendance au service MySQL
- **mysql** — MySQL 8, configuré via variables d'environnement et volume persistant
- **mongo** ■ — MongoDB 6, avec volume persistant (optionnel)

Les identifiants de connexion (utilisateur, mot de passe, nom de la base) doivent être définis comme variables d'environnement dans un fichier .env.

Schéma MySQL

Concevez et implémentez le schéma de base de données dans un fichier 01_schema.sql chargé automatiquement au démarrage du conteneur MySQL. Le schéma doit inclure les tables ci-dessous ainsi que des données de test (quelques genres et films) pour faciliter les essais.

Table	Colonnes principales
users	id, email (unique), password (haché), pseudo, created_at
films	id, titre, realisateur, annee, duree, synopsis, affiche, created_at
genres	id, nom (unique)
film_genre	film_id (FK), genre_id (FK) — table de liaison many-to-many

Authentification

L'application doit proposer un système complet d'inscription et de connexion. Les mots de passe doivent être stockés de façon sécurisée (fonction de hachage PHP). Après connexion, l'identifiant de l'utilisateur est conservé en session. Toute page protégée doit rediriger vers la page de connexion si la session est absente.

Route	Méthode(s)	Description
/register	GET / POST	Formulaire d'inscription
/login	GET / POST	Formulaire de connexion
/logout	GET	Déconnexion et destruction de session

La classe User doit exposer au minimum : création d'un compte, recherche par email, et vérification du mot de passe.

Gestion des films (CRUD)

Implémentez les routes et la logique nécessaires pour lister, consulter, créer, modifier et supprimer des films. Les opérations d'écriture (ajout, modification, suppression) doivent être réservées aux utilisateurs authentifiés.

Route	Méthode(s)	Description	Auth
/films	GET	Liste de tous les films	Non
/films/{id}	GET	Page détail d'un film	Non
/films/add	GET / POST	Formulaire d'ajout	Oui
/films/edit/{id}	GET / POST	Formulaire de modification	Oui
/films/delete/{id}	POST	Suppression d'un film	Oui

*La classe **Film** doit exposer au minimum : récupération de la liste complète, recherche par identifiant, création, modification et suppression.*

Avis utilisateurs — MongoDB ■ Optionnel

MongoDB est particulièrement adapté pour stocker les avis et commentaires : la structure des documents est flexible, les textes sont de longueur variable, et cette approche évite les jointures complexes. Chaque avis doit référencer un film (via son identifiant MySQL) et un utilisateur.

Structure d'un document dans la collection **avis** :

Champ	Type	Description
film_id	Integer	Référence à l'identifiant MySQL du film
user_id	Integer	Référence à l'identifiant MySQL de l'utilisateur
note	Integer (1–5)	Note attribuée au film
commentaire	String	Texte libre de l'avis
created_at	ISODate	Date et heure de création

Fonctionnalités attendues : ajout d'un avis sur la page détail d'un film, affichage de la liste des avis, calcul de la note moyenne (agrégation MongoDB).

Interface graphique ■ Optionnel

Améliorations CSS attendues :

- Design responsive — approche mobile-first
- Cartes films avec effet au survol
- Formulaires stylisés
- Barre de navigation affichant l'état connecté / déconnecté
- Palette de couleurs type cinéma : noir, rouge, or

Améliorations JavaScript attendues :

- Recherche de films en temps réel sans rechargement de page
- Confirmation avant suppression (fenêtre modale)
- Affichage dynamique des genres
- Validation des formulaires côté client

Grille d'évaluation

Critère	Points
PHP orienté objet (classes, propriétés, méthodes)	
Docker fonctionnel	
Schéma MySQL correct	
Authentification complète	
CRUD films opérationnel	
Intégration MongoDB ■	
Interface soignée (CSS / JS) ■	

■ *Critères optionnels — points bonus uniquement*

Bonus — Pour aller plus loin

- ■ Pagination de la liste des films
- ■ Upload d'affiches de films
- ■ Système de favoris
- ■ Tests unitaires

Commandes Docker utiles

Action	Commande
Démarrer les services	`docker-compose up -d`
Voir les logs	`docker-compose logs -f`
Arrêter les services	`docker-compose down`
Reconstruire	`docker-compose up -d --build`

Accès MySQL (CLI)	`docker exec -it allocine-mysql \ mysql -u allocine_user -pallocine_pass allocine_db`
Accès MongoDB (shell)	`docker exec -it allocine-mongo mongosh`

PROPOSITION DE PLANNING

Répartition indicative sur 3 jours

Ce planning est indicatif et peut être adapté selon votre rythme. Les éléments optionnels (■) sont à aborder uniquement si vous avez terminé les parties obligatoires.

JOUR 1 — Fondations et environnement

Horaire	Activité	Livrable attendu
Matin (9h – 12h)	Mise en place de l'environnement Docker (docker-compose.yml, Dockerfile, .env)	Services démarrés et accessibles
Début d'après-midi (13h–15h)	Conception du schéma MySQL (tables, relations, données de test)	Fichier 01_schema.sql fonctionnel
Fin d'après-midi (15h–17h)	Création des classes PHP (User, Film, Genre + connexion PDO)	Classes avec propriétés et méthodes de base

JOUR 2 — Authentification et CRUD films

Horaire	Activité	Livrable attendu
Matin (9h – 12h)	Inscription, connexion, déconnexion (sessions, hachage mot de passe)	Authentification fonctionnelle
Début d'après-midi (13h–15h)	Liste des films et page détail (avec recherche par titre)	Pages publiques opérationnelles
Fin d'après-midi (15h–17h)	Formulaires d'ajout, modification et suppression de films	CRUD complet réservé aux connectés

JOUR 3 — Finition, tests et options

Matin (9h – 11h)	Tests, correction des bugs, revue du code	Application stable et sans erreurs
Milieu (11h – 13h)	■ Intégration MongoDB (avis, note moyenne)	Collection avis fonctionnelle
Après-midi (14h – 16h)	■ CSS et JavaScript (responsive, interactions)	Interface soignée et dynamique

Jalons de livraison

Jalon	Critère de validation	Obligatoire
Fin Jour 1	Docker démarré · Schéma MySQL chargé · Classes PHP créées	Oui
Fin Jour 2	Authentification opérationnelle · CRUD films fonctionnel	Oui
Fin Jour 3	Application stable et testée · Démo présentée	Oui
Bonus Jour 3	Avis MongoDB + Interface CSS / JS	Non (bonus)

A vous de jouer !